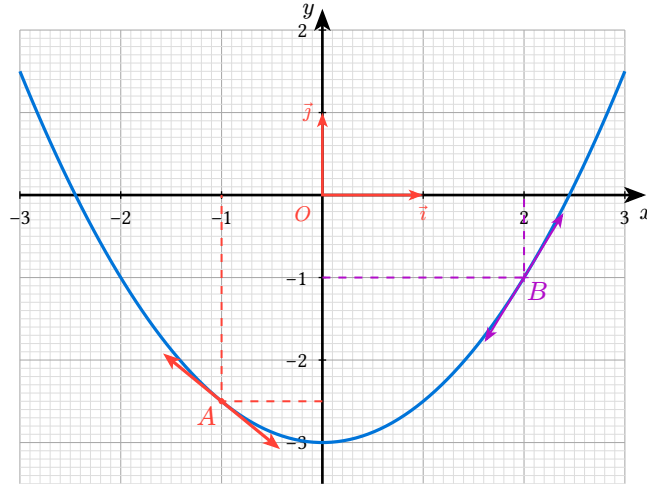


ctz-plot Helper

– Quick reference –



- anchored-basis-vectors()
- anchored-derivative-arrow()
- anchored-lines()
- anchored-point()
- anchored-points()
- draw-mark-shape()
- monplot()
- newFig()
- plot-scale()
- point-marker()
- resolve-origine()
- rotate-point()
- scatter()
- setAxes()
- setExtraStyles()
- setPlot()

Variables

- mesaxes

anchored-basis-vectors

Draws the two basis vectors of (O ; $\text{vec}(i)$, $\text{vec}(j)$), with a length of length DATA units (default 1 = unit vector). Shaft and arrowhead are drawn directly in the canvas frame: no distortion from the frame anisotropy. The two segments form ONE continuous polyline (i) – (O) – (j): no “step” at the origin. To be called OUTSIDE `plot.plot(...)`. Automatically retrieves `sx`, `sy` (computed by `newFig`): nothing to pass again.

- origine (str, array): anchor “plot.” (“plot.O” by default), or (x, y) in data units
- x-length (float, int): length of $\text{vec}(i)$, in data units
- y-length (float, int): length of $\text{vec}(j)$, in data units

- fill (color): color of the arrows and labels
- stroke (auto, stroke): stroke (auto = fill + 1pt)
- mark-scale (float, int): arrowhead size

Parameters

```
anchored-basis-vectors(
  origine: string array,
  x-length: float int,
  y-length: float int,
  fill: color,
  stroke: auto stroke,
  mark-scale: float int
) -> content
```

origine string or array

Anchor “plot.” (“plot.O” by default), or (x, y) in data units.

Default: `"plot.0"`

x-length float or int

Length of vec(i), in data units.

Default: `1`

y-length float or int

Length of vec(j), in data units.

Default: `1`

fill color

Color of the arrows and labels.

Default: `red`

stroke auto or stroke

Stroke (auto = fill + 1pt).

Default: `auto`

mark-scale float or int

Arrowhead size.

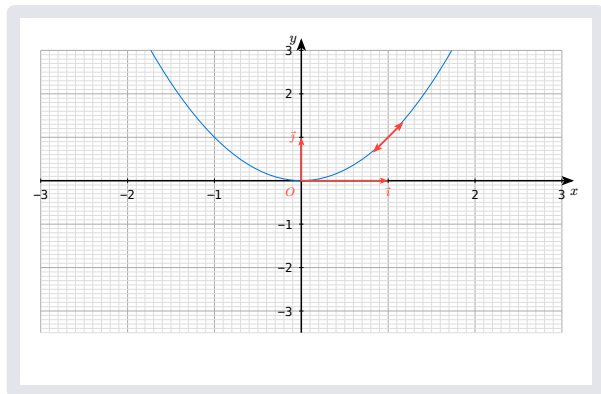
Default: `.5`

anchored-derivative-arrow

Derivative arrow with a FIXED size (cm), with the slope corrected to stay geometrically consistent in an anisotropic frame. To be called OUTSIDE `plot.plot(...)`. Automatically retrieves `sx, sy` (computed by `newFig`): nothing to pass again.

- `origine` (str, array): anchor “plot.”, or (x, y) in data units
- `slope` (float, int): REAL slope in data units (e.g. `spline-dy(...)`)
- `fill` (color): arrow color
- `length` (float, int): FIXED length of the arrow, in cm
- `stroke` (stroke): stroke thickness/color
- `mark-scale` (float, int): arrowhead size

```
#newFig(  
  extra: (sx, sy) => {  
    import draw: *  
    // tangent at point (1, f(1)) with  
    slope 2  
    anchored-derivative-arrow((1, 1), 2,  
    fill: red, stroke: red + 1.2pt)  
  },  
  { plot.add(domain: (-2, 2), x =>  
    calc.pow(x, 2), style: (stroke: blue +  
    0.5pt), samples: 50) },  
)
```



Parameters

```
anchored-derivative-arrow(  
  origine: string array,  
  slope: float int,  
  fill: color,  
  length: float int,  
  stroke: stroke,  
  mark-scale: float int  
) -> content
```

origine string or array

Anchor “plot.”, or (x, y) in data units.

slope float or int

REAL slope in data units (e.g. `spline-dy(...)`).

fill color

Arrow color.

Default: black

length `float` or `int`

FIXED length of the arrow, in cm.

Default: `1`

stroke `stroke`

Stroke thickness/color.

Default: `1pt`

mark-scale `float` or `int`

Arrowhead size.

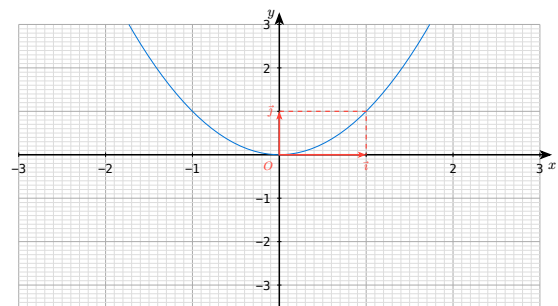
Default: `.6`

anchored-lines

Draws a polyline (successive segments) between several points of the frame (`O ; i, j`) — each point is a “plot.” anchor or (x, y) coordinates in data units, as in `resolve-origine`. The points are converted to canvas coordinates BEFORE being connected, so the stroke stays straight on screen even in an anisotropic frame (unlike a line drawn in data units inside the plot). To be called OUTSIDE `plot.plot(...)`. Automatically retrieves `sx, sy` (computed by `newFig`): nothing to pass again.

- `points` (`str` | `array`): “plot.” anchors and/or (x, y) coordinates, at least two
- `stroke` (`stroke`): line stroke
- `mark` (`none` | `dictionary`): arrowheads at the ends (see `line(mark:)`)

```
#newFig(  
  extra: (sx, sy) => {  
    import draw: *  
    // dashed lines (0 ; f) -- (x ; f) --  
    (x ; 0)  
    anchored-lines((0, 1), (1, 1), (1, 0),  
stroke: (thickness: 0.75pt, dash:  
"dashed", paint: red))  
  },  
  { plot.add(domain: (-2, 2), x =>  
calc.pow(x, 2), style: (stroke: blue +  
0.5pt), samples: 50) },  
)
```



Parameters

```
anchored-lines(  
  ..points: string array,  
  stroke: stroke,  
  mark: none dictionary  
) -> content
```

..points `string` or `array`

“plot.” anchors and/or (x, y) coordinates, at least two.

stroke `stroke`

Line stroke.

Default: black + 1pt

mark `none` or `dictionary`

Arrowheads at the ends (see line(mark:)).

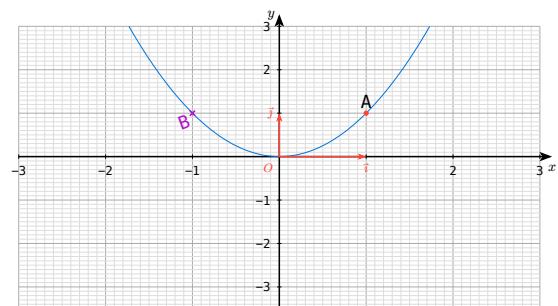
Default: none

anchored-point

Optional marker and/or label at a plot anchor OR at (x, y) coordinates in the frame units (O ; i, j) — see resolve-origine. Automatically retrieves sx, sy (computed by newFig). To be called OUTSIDE plot.plot(...). Each item is independent: if marker or label is none, it is not drawn (both none → nothing at all).

- origine (str, array): anchor “plot.”, or (x, y) in data units
- marker (dictionary, none): (marker-symbol, marker-size, marker-fill, marker-stroke, marker-angle) — see draw-mark-shape ; none = no point
- label (dictionary, none): (label-text, label-distance, label-anchor, label-position, label-rotate, label-styles) ; none = no label

```
#newFig(  
  extra: (sx, sy) => {  
    import draw: *  
    // marker + label (each is optional)  
    anchored-point((1, 1), marker:  
(marker-fill: red), label: (label-text:  
"A", label-position: 90deg))  
    // label only, without a point,  
    rotated 20°  
    anchored-point((-1, 1), marker:  
(marker-symbol: "x", marker-size: 0.06,  
marker-stroke: 1pt+purple), label: (label-  
text: text(fill: purple)[B], label-  
position: -135deg, label-rotate: 20deg))  
  },  
  { plot.add(domain: (-2, 2), x =>  
calc.pow(x, 2), style: (stroke: blue +  
0.5pt), samples: 50) },  
)
```



Parameters

```
anchored-point(  
  origine: string array ,  
  marker: dictionary none ,  
  label: dictionary none  
) -> content
```

origine string or array

Anchor “plot.”, or (x, y) in data units.

marker dictionary or none

Marker dictionary — two accepted vocabularies, see draw-mark-shape ; none = no point:

- internal: (marker-symbol, marker-size, marker-fill, marker-stroke, marker-angle)
- cetz/plot.add style: (mark, mark-fill, mark-stroke, mark-size)

Default: (marker-symbol: "o", marker-size: 0.06, marker-fill: red, marker-stroke: none, marker-angle: 0deg)

label dictionary or none

(label-text, label-distance, label-anchor, label-position, label-rotate, label-styles) ; none = no label.

Default: (

```
  label-text: "",  
  label-distance: 8pt,  
  label-anchor: "center",  
  label-position: 0deg,  
  label-rotate: 0deg,  
  label-styles: (:),
```

)

anchored-points

Draws several points (and, optionally, their labels) in one call. Each item may be either:

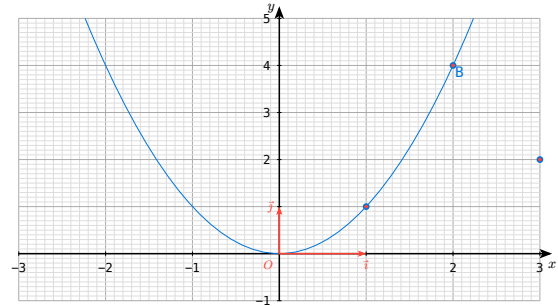
- a bare coordinate (x, y) or a bare anchor “plot.”: no label,
- a pair (origine, label-text): same origine vocabulary as anchored-point, and label-text is either the text/content for THIS point’s label, or none to skip the label for that point.

The two forms can be mixed freely: each item is detected automatically (an item whose first element is itself an array/string is treated as (origine, label-text)). The marker:/label: options are shared by every point (same vocabulary as anchored-point) — label: none (or marker: none) turns off labels (or markers) for ALL points at once, regardless of what each item says. The shared label: options also accept label-styles, a dictionary spread into text(...) to style every label at once (an explicitly styled label-text keeps priority).

- points (array): per point, a bare (x, y) / anchor string, or a pair (origine, label-text) — label-text may be none

- marker (dictionary, none): shared marker options (marker- **or** **cetz mark**: vocabulary), see anchored-point ; none = no markers at all
- label (dictionary, none): shared label options (label-text here is overridden per point, label-styles is spread into text()), see anchored-point ; none = no labels at all

```
#newFig(y-min: -1, y-max: 5,
  extra: (sx, sy) => {
    import draw: *
    anchored-points(
      (1, 1),           // bare point,
no label
      ((2, 4), "B"),    // point with
label
      ((3, 2), none),   // point, label
explicitly disabled
      marker: (mark: "o", mark-fill: red,
mark-stroke: 1pt + blue),
      label: (label-position: -45deg,
label-distance: 6pt, label-styles: (fill:
blue, size: 0.8em)),
    )
  },
  { plot.add(domain: (-3, 3), x =>
calc.pow(x, 2), style: (stroke: blue +
0.5pt), samples: 50) },
)
```



Parameters

```
anchored-points(
  ..points: array,
  marker: dictionary none,
  label: dictionary none
) -> content
```

..points array

Per point: a bare (x, y) / anchor string, or a pair (origine, label-text) — label-text may be none.

marker dictionary or none

Shared marker options (marker- **or** **cetz mark**: vocabulary), see anchored-point ; none = no markers at all.

Default: (marker-symbol: "o", marker-size: 0.06, marker-fill: red, marker-stroke: none, marker-angle: 0deg)

label dictionary or **none**

Shared label options (label-text here is overridden per point, label-styles is spread into text()), see anchored-point ; none = no labels at all.

Default: (

```
    label-text: "",
    label-distance: 8pt,
    label-anchor: "center",
    label-position: 0deg,
    label-rotate: 0deg,
    label-styles: (:),
)
```

draw-mark-shape

Draws the “mark” of a point at a position ALREADY IN CANVAS COORDINATES (x, y in cm). Shape choice shared between point-marker (draws in data units, inside plot.annotate) and anchored-point (draws in cm, outside the plot). Do not call directly in normal use.

- x (float, int): x-coordinate of the point, in canvas units
- y (float, int): y-coordinate of the point, in canvas units
- symbol (str): “o” (circle), “x” (cross), “+” (plus), “square” (square), “triangle”, “diamond” (diamond)
- size (float, int): half-size of the symbol, in the same unit as x, y
- fill (color): fill color (closed shapes) or stroke color (“+”/“x”)
- stroke (none, stroke, color): outline of closed shapes (none = no outline, just fill)
- angle (angle): orientation (relevant for “square”, “triangle”, “diamond”, “+”, “x” — no effect on “o”)

Parameters

```
draw-mark-shape(
    x: float int,
    y: float int,
    symbol: string,
    size: float int,
    fill: color,
    stroke: none stroke color,
    angle: angle
) -> content
```

x float or int

X-coordinate of the point, in canvas units.

y float or int

Y-coordinate of the point, in canvas units.

symbol `string`

“o” (circle), “x” (cross), “+” (plus), “square” (square), “triangle”, “diamond” (diamond).

Default: `"o"`

size `float` or `int`

Half-size of the symbol, in the same unit as x, y.

Default: `0.06`

fill `color`

Fill color (closed shapes) or stroke color (“+”/“x”).

Default: red

stroke `none` or `stroke` or `color`

Outline of closed shapes (none = no outline, just fill).

Default: `none`

angle `angle`

Orientation (relevant for “square”, “triangle”, “diamond”, “+”, “x” — no effect on “o”).

Default: `0deg`

monplot

Wraps `plot.plot(...)`: takes a ready-made options dictionary (typically produced by `setPlot(...)`) and the body that goes INSIDE the plot (your `plot.add(...)`, `plot.add-anchor(...)`, `plot.annotate(...)`). Automatically adds `plot.add-anchor("O", (0, 0))`: you no longer need to do it yourself, the “plot.O” anchor is always available for anchored-basis-vectors(...) etc.

- options (dictionary): options of `plot.plot`, typically produced by `setPlot(...)`
- body (content): content placed in `plot.plot` (`plot.add`, `plot.add-anchor`, `plot.annotate`...)

Parameters

```
monplot(  
    options: dictionary,  
    body: content  
) -> content
```

options `dictionary`

Options of `plot.plot`, typically produced by `setPlot(...)`.

body content

Content placed in `plot.plot` (`plot.add`, `plot.add-anchor`, `plot.annotate`...).

newFig

“All-in-one” function to build a figure: computes `sx`, `sy` automatically (no more need to call `plot-scale` yourself), opens the canvas and applies the axis style, calls `monplot(...)` with the merged options of `setPlot(...)`, and adds, if provided, the content OUTSIDE the plot but INSIDE the canvas (access to `sx`, `sy`). If `reper`: `true` (default), automatically adds the frame (`O ; i, j`) AND hides the “1” graduation on the axes (since it duplicates the `i, j` arrows); if `reper`: `false`, no arrows, and the “1” graduation becomes visible again like the others. An `x-format/y-format` given in `plot`: takes precedence over the one controlled by `reper`: — but do not set both at once for the same key (`x-format` or `y-format`), Typst rejects a duplicate named argument.

- `size` (array): physical dimensions of the plot (width, height), in cm
- `x-min` (float | int): lower bound of the x domain
- `x-max` (float | int): upper bound of the x domain
- `y-min` (float | int): lower bound of the y domain
- `y-max` (float | int): upper bound of the y domain
- `axes` (dictionary): axis style, see `setAxes()`
- `extraStyles` (dictionary): styles of the basis arrows (`i, j`), see `setExtraStyles()`
- `reper` (bool): shows/hides (`O ; vec(i), vec(j)`) and the “1” graduation (see description)
- `plot` (dictionary): override of the `setPlot()` options (e.g. (`x-tick-step`: 0.5))
- `extra` (function, none): (`sx, sy`) => content, drawn after the plot, inside the canvas
- `body` (content): content placed INSIDE `plot.plot` (`plot.add`, `plot.add-anchor`, `plot.annotate`...)

```
#newFig(  
  // content INSIDE plot.plot  
  {  
    plot.add(domain: (-3, 3), x => x * 0, style: (stroke: gray + 0.5pt), samples: 2)  
    plot.add-anchor("p0", (-2, -1))  
  },  
  // content OUTSIDE the plot, inside the canvas (access to sx, sy)  
  extra: (sx, sy) => {  
    import draw: *  
    anchored-point("plot.p0", marker: (marker-fill: red), label: (label-text: "A",  
label-position: 90deg))  
    anchored-basis-vectors(fill: red)  
  },  
)
```

Parameters

```
newFig(  
    size: array ,  
    x-min: float int ,  
    x-max: float int ,  
    y-min: float int ,  
    y-max: float int ,  
    axes: dictionary ,  
    extraStyles: dictionary ,  
    repere: bool ,  
    plot: dictionary ,  
    extra: function none ,  
    body: content  
) -> content
```

size array

Physical dimensions of the plot (width, height), in cm.

Default: (12.0, 6.5)

x-min float or int

Lower bound of the x domain.

Default: -3.0

x-max float or int

Upper bound of the x domain.

Default: 3.0

y-min float or int

Lower bound of the y domain.

Default: -3.5

y-max float or int

Upper bound of the y domain.

Default: 3.0

axes dictionary

Axis style, see setAxes().

Default: [setAxes\(\)](#)

extraStyles dictionary

Styles of the basis arrows (i, j), see setExtraStyles().

Default: `setExtraStyles()`

repere bool

Shows/hides (O ; vec(i), vec(j)) and the “1” graduation ; if false, the origin is labeled “0” (without the red of O).

Default: `true`

plot dictionary

Override of the setPlot() options (e.g. (x-tick-step: 0.5)).

Default: `( : )`

extra function or none

(sx, sy) => content, drawn after the plot, inside the canvas.

Default: `none`

body content

Content placed INSIDE plot.plot (plot.add, plot.add-anchor, plot.annotate...).

plot-scale

Scale factors (canvas units per data unit) of a plot whose physical size and (fixed) domain of both axes are known. Used to convert a real slope into a visual on-screen slope.

- size (array): physical size of the plot (width, height), in cm
- x-domain (array): domain (x-min, x-max) of the x-axis
- y-domain (array): domain (y-min, y-max) of the y-axis

Parameters

```
plot-scale(  
  size: array ,  
  x-domain: array ,  
  y-domain: array  
) -> dictionary
```

size array

Physical size of the plot (width, height), in cm.

x-domain array

Domain (x-min, x-max) of the x-axis.

y-domain array

Domain (y-min, y-max) of the y-axis.

point-marker

Marks a single point (x, y) IN DATA UNITS — to be used INSIDE plot.annotate (cetz-plot then handles the scaling itself). Same vocabulary of shapes as plot.add (mark:), but without its single-point bug. See draw-mark-shape for parameter details.

- x (float, int): x-coordinate of the point, in data units
- y (float, int): y-coordinate of the point, in data units
- symbol (str): “o”, “x”, “+”, “square”, “triangle”, “diamond”
- size (float, int): half-size of the symbol, in data units
- fill (color): fill color or stroke color
- stroke (none, stroke, color): outline (none = no outline, just fill)
- angle (angle): symbol orientation

Parameters

```
point-marker(  
    x: float int,  
    y: float int,  
    symbol: string,  
    size: float int,  
    fill: color,  
    stroke: none stroke color,  
    angle: angle  
) -> content
```

x float or int

X-coordinate of the point, in data units.

y float or int

Y-coordinate of the point, in data units.

symbol string

“o”, “x”, “+”, “square”, “triangle”, “diamond”.

Default: "o"

size float or int

Half-size of the symbol, in data units.

Default: 0.06

fill color

Fill color or stroke color.

Default: red

stroke none or stroke or color

Outline (none = no outline, just fill).

Default: none

angle angle

Symbol orientation.

Default: 0deg

resolve-origine

Resolves origine to a CANVAS position (cm): either an existing anchor (“plot.”), or (x, y) coordinates in the frame units (O ; i, j) — in which case it starts from the “plot.O” anchor and applies sx, sy. Used by anchored-basis-vectors, anchored-point and anchored-derivative-arrow; not meant to be called directly.

- ctx (dictionary): current cetz context (provided by get-ctx)
- origine (str, array): anchor “plot.”, or (x, y) in data units
- sx (float, int): scale factor along x (cm per data unit)
- sy (float, int): scale factor along y (cm per data unit)

Parameters

```
resolve-origine(  
  ctx: dictionary ,  
  origine: string array ,  
  sx: float int ,  
  sy: float int  
) -> array
```

ctx dictionary

Current cetz context (provided by get-ctx).

origine `string` or `array`

Anchor “plot.”, or (x, y) in data units.

sx `float` or `int`

Scale factor along x (cm per data unit).

sy `float` or `int`

Scale factor along y (cm per data unit).

rotate-point

Rotates the point (x, y) by angle around the center (cx, cy).

- cx (float, int): x-coordinate of the rotation center
- cy (float, int): y-coordinate of the rotation center
- x (float, int): x-coordinate of the point to rotate
- y (float, int): y-coordinate of the point to rotate
- angle (angle): rotation angle

Parameters

```
rotate-point(  
  cx: float int,  
  cy: float int,  
  x: float int,  
  y: float int,  
  angle: angle  
) -> array
```

cx `float` or `int`

X-coordinate of the rotation center.

cy `float` or `int`

Y-coordinate of the rotation center.

x `float` or `int`

X-coordinate of the point to rotate.

y `float` or `int`

Y-coordinate of the point to rotate.

angle angle

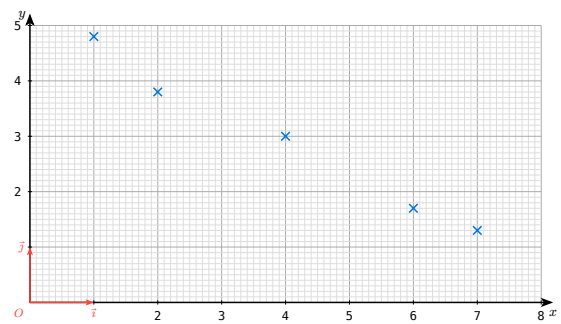
Rotation angle.

scatter

Scatter plot: draws a marker at each point of points. To be used like `plot.add(...)`, INSIDE `plot.plot(...)` — i.e. directly inside `newFig`'s body — coordinates stay in DATA units, `cetz-plot` handles the scaling itself. Thin wrapper around `plot.add(...)`, with a friendlier point-marker vocabulary (`mark-fill/mark-stroke` instead of a raw `mark-style` dictionary).

- `points` (array): list of (x, y) point tuples, in data units
- `mark` (str): marker shape ("o", "x", "+", "square", "triangle", "diamond"...), forwarded to `plot.add(mark:)`
- `mark-fill` (color): marker fill color
- `mark-stroke` (none, stroke, color): marker outline (none = no outline)
- `mark-size` (float, int): marker size, forwarded to `plot.add(mark-size:)`

```
#newFig(x-min : 0, x-max:8, y-min: 0, y-  
max: 5,  
  {  
    scatter(  
      ((1, 4.8), (2, 3.8), (4, 3), (6,  
1.7), (7, 1.3)),  
      mark: "x",  
      mark-stroke: blue,  
      mark-size: 0.2,  
    )  
  },  
)
```



Parameters

```
scatter(  
  points: array,  
  mark: string,  
  mark-fill: color,  
  mark-stroke: none stroke color,  
  mark-size: float int  
) -> array
```

points array

List of (x, y) point tuples, in data units.

mark string

Marker shape ("o", "x", "+", "square", "triangle", "diamond"...), forwarded to `plot.add(mark:)`.

Default: "o"

mark-fill `color`

Marker fill color.

Default: black

mark-stroke `none` or `stroke` or `color`

Marker outline (none = no outline).

Default: `1pt + black`

mark-size `float` or `int`

Marker size, forwarded to `plot.add(mark-size:)`.

Default: `0.1`

setAxes

Axis style dictionary, to pass to `set-style(axes: ...)` (via `newFig(axes: ...)`).

- `grid (stroke)`: main grid style
- `minor-grid (stroke)`: minor grid style
- `shared-zero (content)`: label shown at the common origin of both axes
- `padding (length)`: offset beyond the axes
- `overshoot (length)`: overshoot of the axis arrowheads
- `tick-stroke (stroke)`: color/thickness of major ticks
- `tick-length (float, int)`: length of major ticks
- `tick-minor-stroke (stroke)`: color/thickness of minor ticks
- `tick-minor-length (float, int)`: length of minor ticks

Parameters

```
setAxes(  
  grid: stroke,  
  minor-grid: stroke,  
  shared-zero: content,  
  padding: length,  
  overshoot: length,  
  tick-stroke: stroke,  
  tick-length: float int,  
  tick-minor-stroke: stroke,  
  tick-minor-length: float int  
) -> dictionary
```

grid `stroke`

Main grid style.

Default: `(stroke: gray + 0.4pt)`

minor-grid `stroke`

Minor grid style.

Default: (stroke: gray.lighten(60%) + 0.2pt)

shared-zero `content`

Label shown at the common origin of both axes.

Default: text(size: 8pt, fill: red)[\$0\$]

padding `length`

Offset beyond the axes.

Default: 0pt

overshoot `length`

Overshoot of the axis arrowheads.

Default: 8pt

tick-stroke `stroke`

Color/thickness of major ticks.

Default: black + 0.5pt

tick-length `float` or `int`

Length of major ticks.

Default: 0.1

tick-minor-stroke `stroke`

Color/thickness of minor ticks.

Default: gray + 0.3pt

tick-minor-length `float` or `int`

Length of minor ticks.

Default: 0

setExtraStyles

Extra styles (axis arrowheads), to merge with those of `setAxes()` via `set-style(axes: axes + extraStyles)`.

- `x` (dictionary): style of the arrowhead of the x-axis
- `y` (dictionary): style of the arrowhead of the y-axis

Parameters

```
setExtraStyles(  
  x: dictionary ,  
  y: dictionary  
) -> dictionary
```

x dictionary

Style of the arrowhead of the x-axis.

Default: (mark: (end: "stealth", fill: black))

y dictionary

Style of the arrowhead of the y-axis.

Default: (mark: (end: "stealth", fill: black))

setPlot

Default options of `plot.plot(...)`, as a dictionary. Can be called as is ("generic" values), but `newFig` calls it while injecting ITS OWN size/x-min/x-max/y-min/y-max (and x-format/y-format depending on `reper:`), so that you never have to repeat these values in two places. Any extra parameter not explicitly anticipated (e.g. `y-equal`;) passes through `...more` and is forwarded to `plot.plot`.

- `size` (array): physical size of the plot (width, height), in cm
- `x-min` (float, int): lower bound of the x domain
- `x-max` (float, int): upper bound of the x domain
- `y-min` (float, int): lower bound of the y domain
- `y-max` (float, int): upper bound of the y domain
- `name` (str): name of the plot (for "plot." in `plot.add-anchor`)
- `x-tick-step` (float, int): step between two major graduations on x
- `x-minor-tick-step` (float, int): step between two minor graduations on x
- `y-tick-step` (float, int): step between two major graduations on y
- `y-minor-tick-step` (float, int): step between two minor graduations on y
- `axis-style` (str): axis style ("school-book", etc.)
- `x-format` (function): formatting of the values shown on the x-axis
- `y-format` (function): formatting of the values shown on the y-axis
- `x-label` (content): label of the x-axis
- `y-label` (content): label of the y-axis
- `x-grid` (str, none): vertical grid ("major", "minor", "both" or none)
- `y-grid` (str, none): horizontal grid ("major", "minor", "both" or none)

Parameters

```
setPlot(  
  size: array ,  
  x-min: float int ,  
  x-max: float int ,  
  y-min: float int ,  
  y-max: float int ,  
  name: string ,  
  x-tick-step: float int ,  
  x-minor-tick-step: float int ,  
  y-tick-step: float int ,  
  y-minor-tick-step: float int ,  
  axis-style: string ,  
  x-format: function ,  
  y-format: function ,  
  x-label: content ,  
  y-label: content ,  
  x-grid: string none ,  
  y-grid: string none ,  
  ..more: any  
) -> dictionary
```

size array

Physical size of the plot (width, height), in cm.

Default: (12, 6.5)

x-min float or int

Lower bound of the x domain.

Default: -3

x-max float or int

Upper bound of the x domain.

Default: 3

y-min float or int

Lower bound of the y domain.

Default: -3.5

y-max float or int

Upper bound of the y domain.

Default: 3

name `string`

Name of the plot (for “plot.” in plot.add-anchor).

Default: `"plot"`

x-tick-step `float` or `int`

Step between two major graduations on x.

Default: `1`

x-minor-tick-step `float` or `int`

Step between two minor graduations on x.

Default: `0.1`

y-tick-step `float` or `int`

Step between two major graduations on y.

Default: `1`

y-minor-tick-step `float` or `int`

Step between two minor graduations on y.

Default: `0.1`

axis-style `string`

Axis style (“school-book”, etc.).

Default: `"school-book"`

x-format `function`

Formatting of the values shown on the x-axis.

Default: `v => if v != 1 { text(size: 8pt)[#v] }`

y-format `function`

Formatting of the values shown on the y-axis.

Default: `v => if v != 1 { text(size: 8pt)[#v] }`

x-label `content`

Label of the x-axis.

Default: `text(size: 0.8em)[$x$]`

y-label `content`

Label of the y-axis.

Default: `text(size: 0.8em)[$y$]`

x-grid `string` or `none`

Vertical grid (“major”, “minor”, “both” or none).

Default: `"both"`

y-grid `string` or `none`

Horizontal grid (“major”, “minor”, “both” or none).

Default: `"both"`

..more `any`

Extra parameters forwarded as is to `plot.plot`.

mesaxes `function`

Alias for compatibility with the old name.